UNIVERSITY OF MICHIGAN
Department of Electrical Engineering and Computer Science
EECS 445 — Introduction to Machine Learning
Fall 2025

Project 1 - Predicting Survival in the ICU
Issued: 9/10 – Due: 9/24 at 10:00pm

Introduction

With the rapid advancement of technology, hospitals are creating and collecting vast amounts of data about their patients. These data, stored in the electronic health record, have the potential to impact clinical care in a variety of ways, e.g., in identifying patients at greatest risk of adverse outcomes, in predicting the efficacy of a particular drug, or in matching patients with the best treatments.

In this project, you will be working with a subset of publicly available data collected from patients admitted to an intensive care unit at the Beth Israel Deaconess Medical Center in Boston. You will develop classification tools that automatically identify patients at risk of in-hospital mortality. Such a model could help clinicians target interventions, facilitating better management and improved patient outcomes. We emphasize that these data represent real patient admissions and real outcomes; they have been made available for research and educational purposes by PhysioNet.org. Such datasets are critical to the fundamental advancement of clinical decision support tools.

Getting Started

Throughout this project, we will explore and make extensive use of the scikit-learn package. Coding questions will be highlighted pink, and questions that require written answers will be highlighted yellow.

Please download the Project 1 skeleton code `project1.zip` from Canvas. This zip file contains the full project dataset, and may take several minutes to download.

Requirements:

(a) We will be using Python 3.13.3 throughout this course.

(b) If you do not have Anaconda installed on your machine, please refer to the course's python setup guide on Canvas which can be found under files at `Tutorial/Python Setup Guide.pdf`.

(c) Once Anaconda is installed, run the following command to create a conda environment named `445_p1`: `conda create --name 445_p1 python=3.13.3`. After activating the environment with `conda activate 445_p1`, install required packages with `conda install --file requirements.txt`.

Dataset

these dataset contains data for a total of 12,000 patient admissions. Each admission is associated with a `RecordID` (filename), and a separate csv file under `data/files/`. These csv files contain

timestamped observations for several variables. Outcomes for each patient admission are listed in `data/labels.csv`, in which the first column is `RecordID`. The second column `In-hospital_death` contains binary labels: 1 means the patient died in the hospital, and -1 means the patient survived and was discharged. The third column `30-day_mortality` also contains binary labels: 1 means the patient died within 30 days of discharge, and -1 means the patient survived past 30 days.

For questions 2 to 4 you will use data from the first 2,000 patients to explore several modeling and hyperparameter selection techniques. For the final challenge, you may use data from all 12,000 examples. Note that both outcome labels for the last 2,000 examples have been removed, and we will use these examples as the held-out set for evaluating your final model's performance.

Structure

The script `helper.py` provides functions for reading in the data. The data has already been loaded in the main function of `project1.py`. **Please do not change how the data are read in** (except for the challenge); doing so may affect your results.

The script `project1.py` provides specifications for functions that you will implement:

- `generate_feature_vector(df)`
- `impute_missing_values(X)`
- `normalize_feature_matrix(X)`
- `cv_performance(clf, X, y, metric="accuracy", k=5)`
- `select_param_logreg(X, y, metric="accuracy", k=5, C_range=[], penalties=["L2", "L1"])`
- `select_param_RBF(X, y, metric="accuracy", k=5, C_range=[], gamma_range=[])`
- `plot_weight(X, y, C_range, penalties)`
- Optional: `get_classifier(loss="logistic", penalty=None, C=1.0, class_weight=None, kernel="rbf", gamma=0.1)`
- Optional: `performance(clf_trained, X, y_true, metric="accuracy")`

Resources

All of the libraries that we use are well documented online. Most have two types of documentation: an API reference listing the in-depth technical details of each function/data type in the library, and a user guide with easy-to-follow tutorials on how to use the library at a high level.

- Scikit-learn User Guide and Scikit-learn API Documentation
- NumPy User Guide and NumPy API Documentation
- Pandas User Guide and Pandas API Documentation

Both the scikit-learn user guide and API reference will be very useful throughout this project.

Lecture Content

The sections will use the following main topics from lectures:

1. Feature vectors;
2. Linear models, regularization, model selection, model performance;
3. Ridge regression, regularization, model selection, model performance;
4. Kernels, model selection, model performance;
5. The challenge will use everything you've learned so far.

1 Feature Extraction [20 pts]

For each patient admission, you are given data collected over the first 48 hours of the ICU admission, as a csv file with three columns: `Time`, `Variable` and `Value`. Each row in the csv file represents a single observation. Each observation has an associated timestamp (`Time`) indicating the time of the observation relative to the start of the ICU admission, in hours and minutes. For example, a timestamp of 35:19 means that the associated observation was made 35 hours and 19 minutes after the patient was admitted to the ICU.

There are two main types of variables: time-invariant and time-varying. Their definitions are specified in `config.yaml`. In particular, time-invariant variables are collected at the time the patient is admitted to the ICU. Their associated timestamps are set to 00:00 (thus they appear at the beginning of each patient's record). Unknown values are explicitly encoded as -1. On the other hand, time-varying variables (e.g., heart rate) for a patient might be measured one time, many times or not at all. You will notice that, in addition to being time-invariant or time-varying, a variable could be one of two data types: numeric or categorical. See the documentation for more details regarding the meaning of each variable.

The goal of feature extraction is to transform each patient's raw data into a d-dimensional feature vector for a classifier to learn on. For this project, we will summarize each time-varying variable by taking the maximum observed value over the entire time period. See Figure 1 for an illustration.

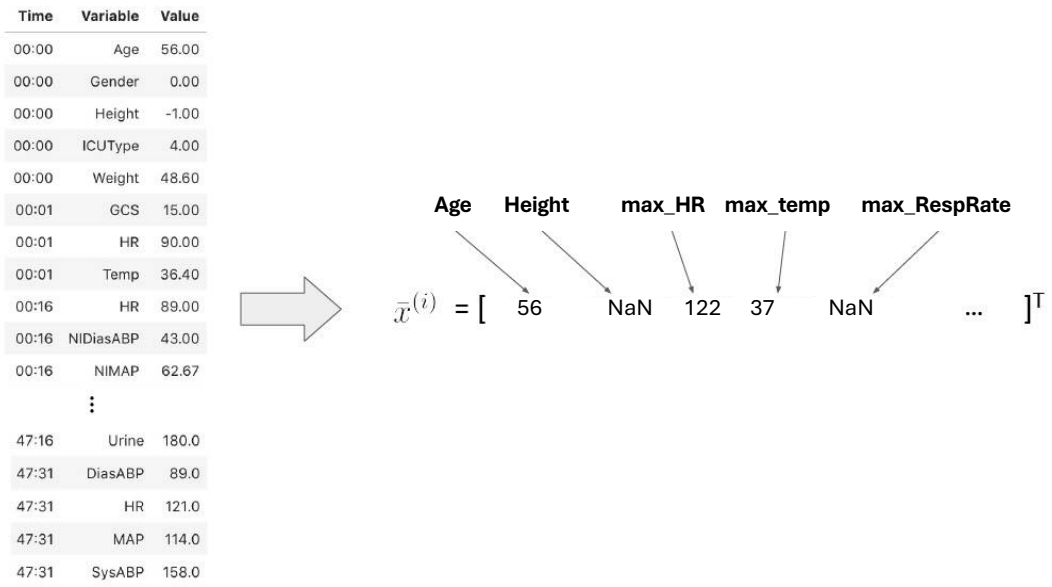

Figure 1: Transforming data into a feature vector. Age and Height are time-invariant variables, each of which is encoded as a separate feature. For this patient, the feature Age has its original value, while the feature Height is `np.nan` since it is unknown (-1). HR, Temp and RespRate are time-varying variables. Here, we will encode each variable with its max observed value. The feature `max_HR` contains the max heart rate measurements, whereas `max_RespRate` is `np.nan` because no respiratory rate measurements were recorded for this patient.

(a) (4 pts) Implement `generate_feature_vector(df)`. The input to this function is a `pd.DataFrame` containing the data for a single patient admission. The output is a python dictionary object, corresponding to a d-dimensional feature vector for this patient (with missing values). The keys of the output dictionary are feature names, and the values are the corresponding feature values. Note, the first five rows of the `pd.DataFrame` input to `generate_feature_vector(df)` contain all of the rows corresponding to time-invariant variables.

For the time-invariant variables, use the raw values. Replace unknown observations (entries denoted by a -1) with undefined (use `np.nan`), and name these features with the variable names from the raw data.

For each time-varying variable, compute the max of all measurements for that variable. If no measurement exists for a variable, the max is also undefined (use `np.nan`). Name these features as `max_{Variable}` for each variable. For example, the variable HR would correspond to the feature with name `max_HR`.

Also answer the following questions (no additional implementation required):

i) (2 pts) Read the documentation on the variable `ICUType`, and reflect on the current feature representation of this variable. How might such a representation impose unintended meaning on the feature? What are the implications of that when using a linear

classifier? How else might you represent this variable (as possibly more than one feature)?

ii) (2pts) Here we only consider the max of the numerical variables. What limitations are associated with this representation? What other summary statistics could be useful?

(b) (4 pts) Implement `impute_missing_values(X)`. Given a feature matrix `X` (where each row corresponds to a patient admission and each column a feature) with missing values, we consider each feature column independently. For each column, we impute missing values by replacing with the mean value of the observed values in that column. Hint: use `np.nanmean()` to compute the mean of an `np.array` with `np.nan` values. Answer the following question (no additional implementation required): What assumptions does this imputation approach make? How else might you handle missing data?

(c) (4 pts) Notice that many of these feature values lie on very different scales. Here, we will address this issue. Implement `normalize_feature_matrix(X)`. Given a feature matrix `X` (where each row corresponds to a patient admission and each column a feature) now without any missing values, we use the following formula to normalize each feature column to have range between 0 and 1:

$$\tilde{x}_d^{(i)} = \frac{x_d^{(i)} - \min}{\max - \min} \text{ where } \min = \min\{x_d^{(j)}\}_{j=1}^n \text{ and } \max = \max\{x_d^{(j)}\}_{j=1}^n,$$

where $x_d^{(i)}$ is the dth feature of the ith datapoint, and $\tilde{x}_d^{(i)}$ represents that same value after normalization. Without feature normalization, regularization can affect some features more than others. Explain how.

(d) (8 pts) Review the implementation of `get_project_data` within `helper.py`. This helper function uses the three functions you implemented in Section 1(a), 1(b), and 1(c). First, it generates a feature vector for each patient, then aggregates them into a feature matrix (features are sorted alphabetically by name), and then performs imputation and normalization with respect to the population (the entire dataset). After all these steps, it splits the data into 80% train and 20% test. Report the name, mean value, and interquartile range for each of the 40 features in the training set `X_train` in a table. In your table, report 4 decimal places. To help check your code for Section 1, we've provided a few rows from the table in Table 1, which should match your results.

Feature	**Mean Value**	**Interquartile Range**
Age	0.6471	0.3378
max_ALP	0.0717	0.0165

Table 1: Subset of the data statistics to check your feature extraction code.

2 Hyperparameter and Model Selection [25 pts]

The skeleton code transforms the patient data into a feature matrix and label vector using the functions you implemented from Section 1, and then splits the data into train and test sets. Training data `X_train, y_train` and test data `X_test, y_test` has already been read in for you. **You will use these data for all parts of Sections 2 to 4.** As mentioned above, you should only have 1,600 training examples and 400 test examples.

We will learn a classifier to separate the *training* data into two classes based on whether or not the patient dies during the hospital stay. The labels in `y_train` are binary labels in $\{-1, 1\}$, where 1 means that the patient has died in hospital.

For the classifier, we will use linear classifiers. We will make use of the `sklearn.linear_model.LogisticRegression` class. In all instances, we will explicitly set the initialization argument `C`. The rest of the initialization arguments are specified when necessary. We will mainly use three methods of the `linear_model.LogisticRegression` class: `fit(X,y)`, `predict(X)` and `decision_function(X)`.

As discussed in lecture, regularized linear models have hyperparameters that must be set by the user. We will select hyperparameters using 5-fold cross-validation (CV) on the training data. We will select the hyperparameters that lead to the **best mean performance** across all five validation folds. The result of hyperparameter selection often depends on the choice of performance measure. Here, we will consider the following performance measures: accuracy, precision, F1-score, AUROC, average precision, sensitivity, and specificity.

Note:

- When calculating the F1-score and precision score, it is possible that a divide-by-zero may occur which raises a warning. Consider how this metric is calculated, perhaps by reviewing the relevant `scikit-learn` documentation.
- Some of these measures are already implemented as functions in the `sklearn.metrics` submodule. Please use `sklearn.metrics.roc_auc_score` for AUROC. You can use the values from `sklearn.metrics.confusion_matrix` to calculate the others. Make sure to read the documentation carefully, we recommend setting `labels=[-1,1]` for a deterministic ordering of the confusion matrix output.
- You should implement helper functions to avoid code duplication. The recommended helpers include: `performance(...)`, which can calculate each performance metric given the true and predicted labels; you can also create a helper function for this function for code simplicity.
- You could also implement another helper function `get_classifier(...)` that returns an instance of the appropriate classifier according to the parameters.

(a) (2 pts) To begin, implement the function `cv_performance(clf, X, y, metric="accuracy", k=5)` as defined in the skeleton code. Here, you will make use of the `fit(X,y)`, `predict(X)` and `decision_function(X)` methods in the `linear_model.LogisticRegression` class. The function returns the mean along with the range (i.e., min and max) k-fold CV performance for

the performance metric passed into the function. The default metric is `"accuracy"`, however your function should work for all of the metrics listed above. Ensure that you are correctly handling cases where you may encounter a division by zero.

When dividing the data into folds for CV, you should try to keep the class proportions (ratio of positive to negative labels) roughly the same across folds: you must implement this function without using the `scikit_learn` implementation of CV. However, you may employ the following class for splitting the data: `sklearn.model_selection.StratifiedKFold()`. **To ensure consistency in grading, do not shuffle points when using this function (i.e., do not set `shuffle=True`).**

It is *strongly recomended* that you implement the `performance(...)` helper function. In the skeleton code, this helper function has an additional input that indicates whether or not to use bootstrap sampling. For `cv_performance(...)`, we will not use bootstrap sampling, but in Section 2(d) we will. You can modify the helper function then.

In your write-up, briefly describe why it might be beneficial to maintain class proportions across folds.

(b) (2 pts) Now implement the `select_param_logreg(X, y, metric="accuracy", k=5, C_range=[], penalties=[])` function to choose a value of C and the type of regularization penalty, for a regularized logistic regression, using the **best mean performance** from 5-fold cross-validation on the training data with specified metric. This function should call the `cv_performance` function that you implemented in part (a), passing in instances of `LogisticRegression(penalty=penalty, C=C, solver="liblinear", fit_intercept=False, random_state=seed)` with a range of values for C chosen in powers of 10 (where $C \in \{10^{-3}, 10^{-2}, 10^{-1}, 10^{0}, 10^{1}, 10^{2}, 10^{3}\}$) and both the ℓ_1 and ℓ_2 regularization penalty.

Note that logistic regression optimizes the objective function

$$J(\bar{\theta}) = z(\bar{\theta}) + C \sum_{i=1}^{n} \text{loss}_{\log}(\bar{x}^{(i)}, y^{(i)}; \bar{\theta}),$$

where $z(\bar{\theta})$ is a regularization penalty, $\text{loss}_{\log}(\bar{x}, y; \bar{\theta})$ is logistic loss, and C is a hyperparameter selected by the user. In your writeup describe the role of C. What do we expect to happen as we increase the value of C?

(c) (8 pts) Using the training data from Section 1 and the functions implemented here, find the best setting for C and type of regularization penalty (i.e., ℓ_1 or ℓ_2) for each performance measure. Report your findings in tabular format with four columns: names of the performance measures, along with the corresponding values of C, regularization penalty and the mean, min and max cross-validation performance. The table should follow the format given below in Table 2.

Performance Measure	C	Penalty	Mean (Min, Max) CV Performance
Accuracy			
Precision			
F1-Score			
AUROC			
Average Precision			
Sensitivity			
Specificity			

Table 2: Cross-validation performance. Report 4 decimal places.

Your `select_param_logreg(X, y, metric="accuracy", k=5, C_range=[], penalties=["L1","L2"])` function should return the best value of C and penalty given a range of values.

Also, in your write-up, describe how the 5-fold CV performance varies with C for specific metrics. If you have to train a final model, which performance measure would you optimize for when choosing C and the regularization penalty? Explain your choice.

(d) (5 pts) Now, using the values of C and regularization penalty that maximize the AUROC measure as found in Section 2(c), train a linear model on the training data `X_train, y_train`. Report the performance of this model on the test data `X_test, y_test` for each performance measure. Specifically, report the median performance of 1,000 bootstrapped test set samples and the empirical 95% confidence interval. Report your results in Table 3.

You can update the `performance(...)` helper function from part (a) to calculate the median performance and the 95% confidence interval from 1,000 bootstrapped samples. This involves sampling with replacement from the input data to form a new dataset with the same size as the original dataset. To get the median performance and 95% confidence interval, we calculate the performance on the new dataset, repeat the procedure 1,000 times, and then calculate the 2.5^{th}, 50^{th}, and 97.5^{th} percentiles.

$C = ?, \quad \text{penalty} = ?$		
Performance Measure	**Median**	**(95% Confidence Interval)**
Accuracy		
Precision		
F1-score		
AUROC		
Average Precision		
Sensitivity		
Specificity		

Table 3: Test Performance for logistic regression with C and penalty from Section 2(c). Report 4 decimal places.

(e) (5 pts) Finish the implementation of the `plot_weight(X, y, C_range, penalties)` function. In this function, you need to find the ℓ_0-norm of $\bar{\theta}$, the parameter vector associated with the

linear model, for each value of C and each type of regularization penalty. Finding out how to get the vector $\bar{\theta}$ from a `clf` object may require you to dig into the documentation. The ℓ_0-norm is given as follows, for $\bar{\theta} \in \mathbb{R}^d$:

$$\|\bar{\theta}\|_0 = \sum_{i=1}^{d} \mathbb{1}(\theta_i \neq 0)$$

where $\mathbb{1}(a \neq 0)$ is 0 if a is 0 and 1 otherwise.

Use the complete training data `X_train, y_train`, i.e., don't use cross-validation for this part. Once you complete the first part of this function, the existing code will plot ℓ_0-norm $\|\bar{\theta}\|_0$ against C and save it to a file. In your write-up, include the produced plot and describe any trends you observe.

(f) (1 pt) Recall that each coefficient of $\bar{\theta}$ is associated with a clinical feature. Using $C = 1.0$ (for consistency with our results), train an ℓ_1-regularized logistic regression model on `X_train, y_train`. Given this model, find the 4 most positive coefficients and the 4 most negative coefficients of $\bar{\theta}$. Find the features that these coefficients correspond to, and report both the coefficients and the corresponding features in Table 4.

Positive Coefficient	**Feature Name**

Negative Coefficient	**Feature Name**

Table 4: Features ranked by coefficient magnitude. Report 4 decimal places.

(g) (2 pts) Consider the features in Table 4. What does it mean for a feature's coefficient to be more positive? How will those features relate to in-hospital mortality as opposed to features with more negative coefficients? What about features with a coefficient of zero?

3 Asymmetric Cost Functions and Class Imbalance [15 pts]

Our training data are *class imbalanced*: the data contain more patients who survived and were discharged than those who died in hospital. As you may have noticed from Section 2, this can lead to skewed performance measures. In such cases, one must be careful when interpreting results. For example, if your data has a class ratio of $4:1$ (4 times as many positive labels as negative labels) an accuracy score of 80% can be achieved trivially.

In this section, you will investigate the objective function of penalized logistic regression, and how we can modify it to deal with class imbalance. The objective function for ℓ_2-regularized logistic regression in scikit-learn uses binary cross-entropy loss and is as follows:

$$J(\bar{\theta}) = \frac{\|\bar{\theta}\|^2}{2} + C\sum_{i=1}^{n} \text{loss}_{ce}(\bar{x}^{(i)}, y'^{(i)}; \bar{\theta}).$$

Binary cross-entropy loss, defined as

$$\text{loss}_{ce}(\bar{x}, y'; \bar{\theta}) = -y' \ln \sigma(\bar{\theta} \cdot \bar{x}) - (1 - y') \ln\big(1 - \sigma(\bar{\theta} \cdot \bar{x})\big),$$

expects labels in $\{0, 1\}$ instead of $\{\pm 1\}$ so we use $y' = \mathbb{1}(y == 1)$. To handle asymmetric data we can modify the original objective in the following way:

$$J(\bar{\theta}) = \frac{\|\bar{\theta}\|^2}{2} + CW_p \sum_{i|y^{(i)}=1} \text{loss}_{ce}(\bar{x}^{(i)}, y'^{(i)}; \bar{\theta}) + CW_n \sum_{i|y^{(i)}=-1} \text{loss}_{ce}(\bar{x}^{(i)}, y'^{(i)}; \bar{\theta})$$

where $\sum_{i|y^{(i)}=1}$ is a sum over all indices i where the corresponding point is positive $y^{(i)} = 1$. Similarly, $\sum_{i|y^{(i)}=-1}$ is a sum over all indices i where the corresponding point is negative $y^{(i)} = -1$.

For all classifiers in this section, use ℓ_2-penalty and $C = 1.0$.

3.1 Binary Cross-Entropy Loss [4 pts]

In homework 1, we trained a logistic regression model using logistic loss with labels $y \in \{\pm 1\}$:

$$\text{loss}_{\log}(\bar{x}, y; \bar{\theta}) = \ln\big(1 + e^{-y\bar{\theta} \cdot \bar{x}}\big).$$

However, scikit-learn uses binary cross-entropy loss with labels $y' \in \{0, 1\}$ instead:

$$\text{loss}_{ce}(\bar{x}, y'; \bar{\theta}) = -y' \ln \sigma(\bar{\theta} \cdot \bar{x}) - (1 - y') \ln\big(1 - \sigma(\bar{\theta} \cdot \bar{x})\big).$$

Show that $\text{loss}_{\log}$ is equivalent to loss_{ce} when $y' = \mathbb{1}(y == 1)$, i.e. the only difference is that they use different labels for negative samples.

3.2 Arbitrary class weights [6 pts]

W_p and W_n are called "class weights" and are built-in parameters in scikit-learn.

(a) (2 pts) If W_p is much greater than W_n, what does this mean in terms of classifying positive and negative points? Describe how this modification may affect the solution.

(b) (2 pts) Create a logistic regression model with the penalty and C specified above. This time, when calling `LogisticRegression`, set `class_weight = {-1: 1, 1: 50}`, or implement and use the `get_classifier` helper function. This corresponds to setting $W_n = 1$ and $W_p = 50$. Train this model on the training data `X_train`, `y_train`. Report the performance for the modified linear model on the test data `X_test`, `y_test` for each metric in Table 5.

$W_n = 1, \quad W_p = 50$		
Performance Measure	**Median**	**(95% Confidence Interval)**
Accuracy		
Precision		
F1-Score		
AUROC		
Average Precision		
Sensitivity		
Specificity		

Table 5: Test performance with class weights. Report 4 decimal places.

(c) (2 pts) Compared to your results in Section 2(d), which performance measures are affected the most by the new class weights? Which performance measures are least affected? Why do you suspect this is the case?

3.3 The ROC curve [5 pts]

(a) (2 pts) Given the above results, we are interested in investigating the AUROC metric more. First, provide a plot of the ROC curves with labeled axes, using $C = 1.0$, for both $W_n = 1, W_p = 1$ and $W_n = 1, W_p = 5$. Put both curves on the same set of axes. Make sure to label the plot with the corresponding class weights.

(b) (3 pts) **Test your understanding:** You've seen that class weights can affect some measures of classifier performance. Suppose you've been given a classifier that achieves reasonable AUROC but poor sensitivity. In your write-up, briefly propose an *alternative* method for "fixing" a classifier that has been **already** trained on imbalanced data. That is, what could you do to create a classifier that achieves better sensitivity *without* changing the class weights?

4 Kernels [20 pts]

Thus far, we have only trained models linear in the space of input features. In this part of the project, we will explore learning models with radial basis function (RBF) kernels. But first, we introduce a new class `sklearn.kernel_ridge.KernelRidge`. While kernelized logistic regression

is possible, we will make use of `KernelRidge` in sklearn for kernelized ridge regression. From the KernelRidge documentation, we see that `KernelRidge` uses ridge regression to learn a linear function in the space induced by the chosen kernel and the data. In Section 4.1, we will use `kernel = "linear"` to compare ridge and logistic regression. In Section 4.2, we will use `kernel = "rbf"` to explore use of RBF kernels with ridge regression.

4.1 Comparing Ridge and Logistic Regression [8pts]

(a) (3 pts) Consider the 1-dimensional toy dataset $S_6 = \left\{(x^{(i)}, y^{(i)})\right\}_{i=1}^{6}$ plotted in Figures 2 and 3. We will compare two linear models trained without regularization or feature maps, but with offsets. First, plot the decision boundary for logistic regression in Figure 2. Second, for linear regression with squared loss plot the prediction $f(x; \bar{\theta}) = \theta_1 x + \theta_0$ and the thresholded decision boundary in Figure 3.

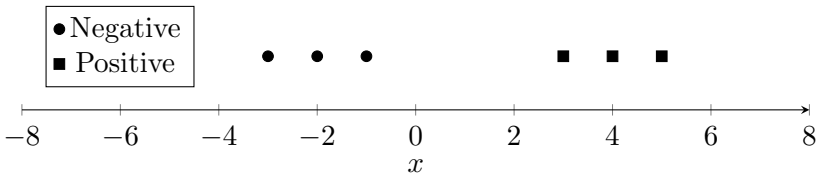

Figure 2: 1D scatter plot of S_6.

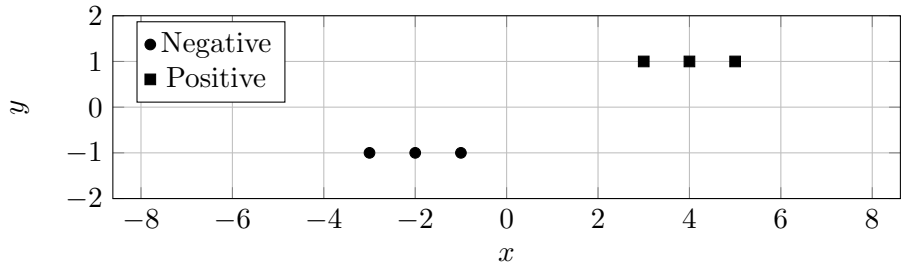

Figure 3: 2D Scatter plot of S_6.

(b) (5 pts) Now compare the empirical performance of a linear model trained using `LogisticRegression` with `KernelRidge`. Use the following to create your models with $C = 1.0$: `LogisticRegression(penalty="L2", C=C, fit_intercept=False, random_state=seed)` and `KernelRidge(alpha=1/(2*C), kernel="linear")`. Use all of the `X_train`, `y_train` to fit each model. Report the test performance obtained by applying the learned models to the test data `X_test`, `y_test`, in terms of the performance measures shown in Table 6. Note that the predictions output by `predict()` in `KernelRidge` are continuous valued. To convert to binary predictions, you will have to threshold the values at 0. That is, all predictions ≥ 0 are mapped to $+1$ and -1 otherwise. Do you observe a large difference in performance between the two approaches? Why or why not?

$C = 1, \quad \text{penalty} = \ell_2$				
Metric	**Logistic Regression**		**Ridge Regression**	
	Median	**(95% Confidence Interval)**	**Median**	**(95% Confidence Interval)**
Accuracy				
Precision				
F1-Score				
AUROC				
Average Precision				
Sensitivity				
Specificity				

Table 6: Testing cross-validation performance for kernel ridge and logistic regression. Report 4 decimal places.

4.2 RBF Kernel [12 pts]

While the prior models are performing well, they're still linear in the data, meaning that they might not be complex enough to learn the true relation. One method to add complexity to the model is to use feature maps, or more efficiently, kernel functions. In this section, we'll implement a ridge regression model using the radial basis function (RBF) kernel, defined as

$$K(\bar{u}, \bar{v}) = \exp\big(-\gamma\|\bar{u} - \bar{v}\|^2\big),$$

where $\gamma \geq 0$ is a hyperparameter that controls the complexity of the kernel.

(a) (0 pts) Implement `select_param_RBF(X, y, metric="accuracy", k=5, C range=[], gamma_range=[])` based on the specification. This function returns the best values of C and γ given a range of values, optimizing for the metric based on cross-validation performance.

(b) (4 pts) Now that we can use Ridge Regression in our classification task, let's add the RBF kernel. Using the training data from question 1 and the functions implemented earlier, report the cross-validation AUROC performance (mean, min, and max) for a fixed $C = 1.0$ and a range of γ values from $\{0.001, 0.01, 0.1, 1, 10, 100\}$. Report your findings and explain the effect γ has on **cross-validation performance** in Table 7.

γ	**Mean (Min, Max) CV Performance**
0.001	
0.01	
0.1	
1.0	
10	
100	

Table 7: Training cross-validation performance for γ. Report 4 decimal places.

(c) (4 pts) Now, using the values from $C \in \{0.01, 0.1, 1.0, 10, 100\}$ and $\gamma \in \{0.01, 0.1, 1, 10\}$ that maximize the AUROC measure, train a non-linear model using KernelRidge and an RBF

kernel on the training data `X_train, y_train`. Report the **test performance** of this model on the test data `X_test, y_test` for each metric in Table 8.

$C = ?, \quad \gamma = ?$		
Performance Measure	**Median Performance**	**(95% Confidence Interval)**
Accuracy		
Precision		
F1-Score		
AUROC		
Average Precision		
Sensitivity		
Specificity		

Table 8: Test performance. Report 4 decimal places.

(d) (4 pts) Is it possible to report the weight associated with each feature in the final model? Why or why not?

5 Challenge [20 pts]

Now, a challenge: in the previous problems, we explored a basic pipeline for extracting features and training a binary classifier. For this challenge, you will consider a set of 10,000 patients. We have already prepared a held-out test set `X_heldout` for this challenge. Your goal is to train a binary classifier using the `LogisticRegression` class to predict 30-day mortality, i.e., whether a patient will survive for at least 30 days post-admission to the ICU.

Note that the class balance in this training set approximately matches the class balance in the held-out set. Also note that, given the size of the data and the feature matrix, training may take several minutes.

In order to attempt this challenge, we encourage you to apply what you have learned about linear models and hyperparameter selection and consider the following extensions:

(a) **Try different feature engineering methods**. The models we have used so far are simplistic. There are other methods to extract different features from the raw data, such as:

- Treat numerical and categorical variables differently
- Consider other summary statistics for numerical variables
- Break up the 48 hours into sub-periods (e.g., two 24-hour windows)
- Use only a subset of the observed data

(b) **Additional preprocessing steps**. In particular, you can:

- Handle missing values differently
- Other feature scaling and transformation techniques

Once you have trained your classifier, you need to provide both the binary predictions (`y_label`) and the real-valued risk scores (`y_score`) of the held-out test set for the submission to be graded. You can cast the output of `clf.predict` to an integer to generate the `y_label` and the output of `clf.decision_function` to generate the `y_score`.

You will have to save the output of your classifier into a `csv` file using the helper function `save_challenge_predictions(y_label, y_score, uniqname)` we have provided. The base name of the output file must be your uniqname followed by the extension `csv`. For example, the output filename for a user with uniqname `foo` would be `foo.csv`. This file will be submitted according to the instructions at the end of this document. You may use the file `test_output.py` to ensure that your output has the correct format. To run this file, simply run `python test_output.py -i uniqname.csv`, replacing the file `uniqname.csv` with your generated output file.

In your write-up, explain the details of your feature extraction and your method of hyperparameter tuning. Also include a 2×2 confusion matrix of the trained classifier applied to the data obtained from `X_challenge` and `y_challenge`.

We will evaluate your performance in this challenge based on two components:

(a) Effort [10 pts]: We will evaluate how much effort you have applied to attempt this challenge based on your write-up and code. **Ensure that both are present.** Within your write-up, you must provide discussions of the choices you made when designing the following components of your classifier:

- Feature engineering
- Hyperparameter selection
- Any techniques you used that go above and beyond current course material

(b) Test Scores [10 pts]: We will evaluate your classifier based on AUROC and F1-score. Both metrics will be considered during grading, so try optimizing both when building your classifier.

REMEMBER to submit your project report by 10:00 pm ET on September 24, 2025 to Gradescope entitled "Project 1 - Write Up".

Submit your code as an appendix to Gradescope entitled **"Project1 - Code Appendix"**. Your submitted code should include any `.py` or `.ipynb` file that you created or modified for this project. Although we won't grade your project based on this appendix, **you will receive 0 points for the whole project if you don't submit the code appendix**.

Submit your `uniqname.csv` file containing the label predictions for the heldout data to the Gradescope assignment entitled **"Project 1 - Challenge Submission".**